

Oleksii Odarchuk

Backend Developer (Go) · Creator of rombik and the Piton language



Lviv, Ukraine +380 68 645 61 42 me@ishawyha.dev
@NeShawyha github.com/OlexiyOdarchuk ishawyha.dev

Backend engineer (Go) with a product mindset and half a year of commercial experience. 1st at BEST Hackathon 2026, 3rd at Mate Hackathon, invited to DOU Day 2026. From hackathon MVPs to production Go services.

EXPERIENCE

Backend Developer (Go)

2026 — present

Skysservice

- Building and maintaining **architecturally complex Go microservices** in a production system: service design, inter-service communication, work under real-world load.

WINS & HACKATHONS

1st Place — Lead Backend & Sensor Fusion

April 18–19, 2026

BEST Hackathon 2026 — GPS-less Drone Trajectory

- Algorithm to **reconstruct a UAV's trajectory without GPS** from IMU/barometer logs. Sensor fusion in **Go + C**: Go for the pipeline & integration, C for the hot numerical kernels.
- 1st place among top teams** — judges singled out the architectural resilience: modular split telemetry / integration / simulation let us swap datasets without rewriting.

3rd Place — Backend Engineer

March 14, 2026

Mate Hackathon (50 participants, 10 teams)

- Designed and built a Go backend from scratch in **7 hours** for an AI-powered MarTech tool that automates competitor-ad analysis.
- Dual AI pipeline: GPT-4o for marketing trigger analysis + Nano Banano 2 (fal.ai) for creative generation; backend routed all AI calls to cloud APIs under hackathon load. Beat teams with 5+ years of experience.

Security Researcher → Job Offer

2026

College Blockchain System “Student Hryvnia”

- Discovered and exploited a critical vulnerability: the **backend never checked transaction amounts** (only the frontend) → arbitrary negative-value transfers. Reported responsibly — received a job offer.
- Reverse-engineered the official JS miner via Webpack bundles — then a faster Go replacement (SHMiner — p. 2).

DOU Day 2026 — Student with High Potential

Personally invited by the organisers with a free ticket — a dedicated track for promising student developers.

TECH STACK

Languages: **Go** **Python** **C**

TypeScript

Backend: **Gin** **Fiber** **REST**

WebSockets **Goroutines** **JWT**

OAuth 2.0

Databases: **PostgreSQL** **Redis**

ClickHouse

Frontend: **Svelte** **Wails**

Tailwind

AI / APIs: **GPT-4o** **Fal.AI**

DevOps: **Docker** **Linux (Arch)**

Git **TinyGo** **Bruno** **Typst**

EDUCATION & LANGUAGES

Network Technologies — IT

College NULP, 2nd year

2024 — present

IT Class — Science Lyceum, Rivne

2022 — 2024

Ukrainian — native · **English** — B1

PROJECTS

◆ rombik — Flowchart Generator (SaaS)

[website](#) → [API](#) →

My first full-fledged commercial SaaS · code in 10 languages → a DSTU 19.701-90 / ISO 5807 flowchart or structogram

Designed and run entirely by me. A **Go engine** — a compiler pipeline (parser → IR → layout → render) on **tree-sitter** for **10 languages** (Python, JavaScript, TypeScript, C, C++, C#, Java, Go, PHP, Pascal); flowcharts and Nassi-Shneiderman structograms, pixel-for-pixel to DSTU 19.701-90 / ISO 5807, byte-exact golden tests. Web editor (pan/zoom, lasso-splitting), 8 formats (SVG/PNG/PDF/Word·Visio·draw.io as native shapes/Typst/Excalidraw). **Billing runs on my own go-monobank-sdk** (HMAC webhooks, grants, receipts). Compiler craft carried over from my language **Piton**. Public **API**, **CLI**, **MCP server**, an AI skill and describe→flowchart via AI. Free watermarked PNG, a clean export needs an account.

[Go](#) · [Gin](#) · [tree-sitter](#) · [API](#) · [CLI](#) · [MCP](#) · [go-monobank-sdk](#) · [PostgreSQL](#) · [DSTU](#)

OPEN SOURCE SDK

go-monobank-sdk — [github](#) → Go SDK for every monobank API

Full coverage of **personal**, **provider** (with monoКЕП), **business** for legal entities, **acquiring** (subscriptions, monopay button, split-receivers, T2P) and **Installment Purchases**. ECDSA webhook verification with auto-rotating keys, drop-in `http.Handler`, typed money/currency/MCC, HMAC-SHA256 for installment callbacks, a fake monobank server for tests, and a separate OpenTelemetry sub-module.

[Go](#) · [ECDSA](#) · [HMAC-SHA256](#) · [OpenTelemetry](#) · [iter.Seq2](#)

SDK CONSUMER

monokasa — Telegram [github](#) → Ticket-Selling Bot

Sells tickets to a single show through a monobank jar and issues PDFs with QR codes scanned at the venue entrance. Live consumer of my own **go-monobank-sdk**: webhook auto-registers at startup, reservations get a 15-minute hold, and show-time reminders fire at-most-once even across restarts.

[Go](#) · [SQLite](#) · [telebot](#) · [PDF + QR](#) · [go-monobank-sdk](#)

1000× FASTER

SHMiner — Desktop [github](#) → Crypto Miner

Cross-platform desktop miner built with **Wails (Go + Svelte/TypeScript)**. Rewrote SHA-256 hashing from JS to Go with a goroutine worker pool — achieved **1000× faster hashrate** than the official client on identical hardware. AES-encrypted wallet storage, real-time WebSocket dashboard, multi-wallet, zen mode.

[Go](#) · [Svelte](#) · [TypeScript](#) · [Wails](#) · [WebSockets](#) · [Goroutines](#)

150+ USERS

AbitAssistant Bot [github](#) →

Telegram bot helping Ukrainian students track university admission rankings. Reverse-engineered closed third-party APIs to fetch live competition data. **150+ active users in first year**, marketing campaign planned for 2026.

[Python](#) · [Aiogram](#) · [PostgreSQL](#) · [BeautifulSoup](#) · [Pandas](#) · [Docker](#)

LinkShortener Bot [github](#) →

Telegram URL shortener with QR generation and geo analytics. PostgreSQL for storage, Redis for redirect caching, ClickHouse for real-time click analytics (country, city, platform). Docker, GeolIP2.

[Go](#) · [PostgreSQL](#) · [Redis](#) · [ClickHouse](#) · [Docker](#) · [Telegram API](#)

MY OWN LANGUAGE

Piton — My own [github](#) → [docs](#) → programming language

Ukrainian transliterated language + interpreter, from scratch in **Go without yacc/bison**: recursive descent parser, tree-walking evaluator, Turing-complete. Flowcharts via a built-in **D2**-based engine. Interpreter compiled to **WebAssembly via TinyGo** (220 KB).

[Go](#) · [TinyGo](#) · [WebAssembly](#) · [D2](#) · [Nix](#)